Lezione 8: Verifica del software, automazione e LLM in Lean

Stefano M. Nicoletti

1 Lezione 8: Verifica del software, automazione e LLM in Lean

Registrazione video della Lezione 8

1.1 Scopo della lezione

Iniziamo dai comandi con cui Lean suggerisce una dimostrazione o risolve un goal automaticamente. Li proviamo su alcuni esempi, poi li usiamo per verificare un algoritmo che rovescia una lista.

Nella lezione precedente abbiamo introdotto brevemente le liste come tipo induttivo. Riprendiamo quella definizione per capire il ruolo del parametro `α` e definire concatenazione e rovesciamento. Dimostriamo per induzione che rovesciare due volte restituisce la lista iniziale. Dopo le dimostrazioni sulle liste, vediamo come usare `decide` e `omega` e come chiedere aiuto a un LLM, controllando con Lean il codice proposto.

1.2 Altre forme di automazione

Le varianti con `?` cercano una dimostrazione e mostrano nell'Infoview un comando suggerito. Quando chiudono il goal, possiamo sostituirle con quel comando.

Vogliamo concludere `a < c` dalle assunzioni `a < b` e `b < c`. `exact?` cerca nella libreria un risultato applicabile e prova a fornirgli gli argomenti necessari.

```
example (a b c : Nat) (hAB : a < b) (hBC : b < c) : a < c := by
  exact?
```

Vogliamo costruire una congiunzione da due assunzioni e un'implicazione. `apply?` cerca un risultato da applicare e prova a dimostrarne le premesse. In questo esempio chiude il goal, ma in altri casi può lasciare premesse da dimostrare.

```
example (P Q R : Prop) (hP : P) (hQ : Q) (hQR : Q → R) : P ∧ R := by
  apply?
```

Vogliamo `P ∧ (Q ∨ R)`, sapendo che `P` e `Q` sono vere. `simp?` usa queste assunzioni per semplificare i connettivi e propone le regole sufficienti.

```
example (P Q R : Prop) (hP : P) (hQ : Q) : P ∧ (Q ∨ R) := by
  simp? [hP, hQ]
```

Vogliamo `Q` dalle assunzioni `P ∨ Q`, `P → R` e `¬R`. `grind?` combina questi fatti. Il caso `P` porta a `R`, in contraddizione con `¬R`, quindi resta il caso `Q`.

```
example (P Q R : Prop) (hPQ : P ∨ Q) (hPR : P → R) (hNotR : ¬R) : Q := by
  grind?
```

Qui il suggerimento è `grind only`, senza una lista di lemmi. Le assunzioni `hPQ`, `hPR` e `hNotR` sono già disponibili nel contesto del goal. Possiamo giustificare la conclusione distinguendo i due casi di `hPQ`: da `P` otteniamo una contraddizione, mentre nel caso `Q` abbiamo proprio ciò che vogliamo dimostrare. `grind` trova una dimostrazione automaticamente, ma il suggerimento non elenca tutti i suoi passaggi interni. `only` esclude i teoremi registrati per `grind`, lasciando disponibili le assunzioni locali e i suoi metodi di ragionamento. Con `simp only [lemma]`, invece, fornivamo una regola con cui semplificare. Senza quel lemma, `simp only` non conosce automaticamente la proprietà della nostra funzione. Può comunque eseguire semplificazioni di base, ma non svolge la stessa ricerca di `grind`.

La ricerca di `exact?` e `apply?` usa le dichiarazioni disponibili nell'ambiente, quindi importazioni e versione di Lean possono cambiare i suggerimenti. Gli esempi qui sono controllati con Lean 4.29.1.

`simp` usa uguaglianze ed equivalenze per semplificare le espressioni. `grind` cerca anche di ricavare nuovi fatti dalle assunzioni e di combinarli. Può distinguere casi, applicare una regola universale a un oggetto e usare le uguaglianze: da `a = b`, per esempio, può ricavare `f a = f b`. Quest'ultimo passaggio si chiama congruenza. Impiega inoltre solver, cioè procedure specializzate per problemi come quelli aritmetici. Questo modo di combinare ragionamento logico e procedure specializzate si ispira ai solver SMT. Se la ricerca fallisce, il goal potrebbe comunque essere dimostrabile con un'altra strategia.

1.3 Test e verifica formale

Un test esegue il programma su input scelti e confronta il risultato con quello atteso. Può rivelare un errore, mentre il superamento del test riguarda gli input esaminati. Con la verifica formale esprimiamo una proprietà e dimostriamo che vale per ogni input compreso nell'enunciato. Possiamo usare entrambi: i test ci aiutano a controllare esempi concreti, la dimostrazione giustifica la proprietà generale.

1.4 Un tipo induttivo per le liste

Definiamo una lista senza usare il tipo `List` della libreria. Il parametro `α` rappresenta il tipo degli elementi contenuti. La stessa definizione può quindi essere usata per liste di naturali, stringhe o altri oggetti.

```
inductive Lista α( : Type) where
  | vuota
  | cons (testa : α) (coda : Lista α)
```

```
open Lista
```

Una `Lista α` è costruita in due modi. `vuota` non contiene elementi. `cons x xs` contiene l'elemento `x` davanti alla lista `xs`. La coda `xs` è a sua volta una lista e costituisce il dato più piccolo sul quale possono procedere ricorsione e induzione.

Per esempio, nella lista con Ada, Ipazia ed Eva, la testa è Ada e la coda contiene Ipazia ed Eva. Se consideriamo poi questa coda, troviamo Ipazia come testa ed Eva nella nuova coda. Dopo Eva rimane la lista vuota. Una funzione ricorsiva sulle liste ripete il proprio lavoro sulle code successive, fino a raggiungere la lista vuota.

Il parametro `α` rende la definizione polimorfa: possiamo scegliere il tipo degli elementi mantenendo gli stessi costruttori. `Lista Naturale` è il tipo delle liste dei naturali definiti nelle lezioni precedenti, mentre `Lista String` è il tipo delle liste di stringhe.

Per esempio, possiamo costruire una lista con tre stringhe:

```
def listaEsempio : Lista String :=
  .cons "Ada" (.cons "Ipazia" (.cons "Eva" .vuota))
```

1.5 Concatenazione e rovesciamento

Per rovesciare una lista ci serve collocare un elemento in fondo a un risultato già costruito. Definiamo prima la concatenazione. La funzione analizza la prima lista. Se questa è vuota restituisce la seconda. Nel caso `cons`, conserva la testa e concatena ricorsivamente la coda.

```
def concatena α{ : Type} (xs ys : Lista α) : Lista α :=
  match xs with
  | .vuota => ys
  | .cons x xs => .cons x (concatena xs ys)
```

Le parentesi graffe in `{α : Type}` rendono `α` un argomento implicito. Quando scriviamo `concatena xs ys`, Lean ricava il tipo degli elementi dai tipi di `xs` e `ys`. Non dobbiamo quindi passare manualmente `α` a ogni chiamata.

La computazione segue la prima lista. Se `xs` è `vuota`, non ci sono elementi da conservare e il risultato è `ys`. Nel ramo `cons x xs`, il nome `xs` indica la coda della lista che stiamo esaminando. La chiamata `concatena xs ys` concatena questa coda con `ys`, poi `cons x` rimette la testa davanti al risultato. La funzione richiama dunque sé stessa, ma su una lista con un elemento in meno. Per esempio, usando una notazione illustrativa con parentesi quadre:

```
concatena [zero, uno] [due]
= zero :: concatena [uno] [due]
= zero :: (uno :: concatena [] [due])
= zero :: (uno :: [due])
= [zero, uno, due].
```

Qui `[]` rappresenta `vuota` e `x :: xs` rappresenta `cons x xs`.

L'algoritmo `rovescia` segue la stessa struttura induttiva. La lista vuota resta vuota. Nel caso `cons x xs`, rovesciamo la coda `xs` e collochiamo `x` in fondo mediante `concatena`.

```
def rovescia α{ : Type} (xs : Lista α) : Lista α :=
  match xs with
  | .vuota => .vuota
  | .cons x xs => concatena (rovescia xs) (.cons x .vuota)
```

Anche `rovescia` è polimorfa. Nel caso `cons x xs`, la chiamata ricorsiva rovescia prima la coda. La lista con il solo elemento `x` viene poi concatenata in fondo. Per capire questo passaggio, prendiamo `[zero, uno]`: la testa è `zero` e la coda è `[uno]`. Rovesciamo `[uno]`, ottenendo ancora `[uno]`, poi aggiungiamo `zero` in fondo:

```
rovescia [zero, uno]
= concatena (rovescia [uno]) [zero]
= concatena (concatena (rovescia []) [uno]) [zero]
= concatena (concatena [] [uno]) [zero]
= concatena [uno] [zero]
= [uno, zero].
```

Per calcolare `rovescia [zero, uno]` ci serve prima il risultato di `rovescia [uno]`, e per quest'ultimo ci serve `rovescia []`. Qui la ricorsione si ferma: il risultato sulla lista vuota è già stabilito dalla definizione. Possiamo allora completare le concatenazioni, prima ottenendo `[uno]` e poi `[uno, zero]`.

La chiamata ricorsiva avviene sempre sulla coda, che contiene un costruttore `cons` in meno. Poiché una lista ha un numero finito di elementi, raggiungiamo il caso `vuota`.

1.6 Formulare una proprietà di correttezza

Una proprietà dell'algoritmo è che rovesciare due volte restituisca la lista iniziale:

```
lecture08_rovescia_rovescia {α : Type} (xs : Lista α) :
  rovescia (rovescia xs) = xs
```

L'enunciato deve valere per ogni tipo `α` e per ogni lista `xs : Lista α`. Esprime una proprietà del rovesciamento. Più avanti vedremo perché questa proprietà, da sola, non basta a specificare il comportamento dell'algoritmo.

Anche nelle dichiarazioni dei teoremi, `{α : Type}` introduce un parametro implicito. Il teorema vale per ogni tipo di elementi. Quando lo usiamo con una lista `xs`, Lean ricava `α` dal tipo della lista: se `xs : Lista String`, sceglie `String`. Le graffe indicano che normalmente possiamo omettere questo argomento. Con `(xs : Lista α)` dichiariamo invece un argomento esplicito, la lista alla quale applicare il teorema.

Per dimostrare questa proprietà useremo alcuni risultati su `concatena` e sul comportamento di `rovescia` rispetto alla concatenazione. Li dimostriamo prima della proprietà finale e osserviamo, nei goal intermedi, quando ciascuno diventa necessario.

1.7 Proprietà della concatenazione

Concatenare la lista vuota a destra non modifica una lista. Poiché `concatena` analizza il primo argomento, dimostriamo il risultato per induzione su `xs`.

```
theorem lecture08_concatena_vuota_destra
    α{ : Type} (xs : Lista α) :
    concatena xs .vuota = xs := by
  induction xs with
  | vuota =>
      rfl
  | cons x xs ih =>
      simp [concatena, ih]
```

Nel caso `vuota`, `concatena vuota vuota` si riduce direttamente a `vuota`. Nel caso `cons x xs`, la definizione conserva `x` e concatena la coda con la lista vuota. L'assunzione induttiva ci dice proprio che `concatena xs vuota = xs`. Usando questa uguaglianza, otteniamo la stessa lista `cons x xs` sui due lati. Il comando `simp [concatena, ih]` permette a Lean di usare la definizione e l'assunzione induttiva per chiudere il goal.

Questa uguaglianza elimina una concatenazione con la lista vuota e produce una forma più semplice. Possiamo quindi aggiungerla al simp-set locale:

```
attribute [local simp] lecture08_concatena_vuota_destra
```

Ci serve inoltre l'associatività della concatenazione. Facciamo induzione su `xs`: la sua forma determina il risultato di `concatena xs ys`, che a sua volta permette di proseguire nella concatenazione esterna a sinistra. Nel passo `cons`, entrambe le espressioni conservano la stessa testa `x`. L'assunzione induttiva permette di uguagliare le concatenazioni delle code.

```
theorem lecture08_concatena_assoc
    α{ : Type} (xs ys zs : Lista α) :
    concatena (concatena xs ys) zs =
      concatena xs (concatena ys zs) := by
  induction xs with
  | vuota =>
      rfl
  | cons x xs ih =>
      simp [concatena, ih]
```

Non aggiungiamo invece l'associatività al simp-set. Questa uguaglianza cambia il raggruppamento, ma non indica in generale una forma più semplice. La nominiamo soltanto nei comandi che ne hanno bisogno.

1.8 Rovesciare una concatenazione

Quando rovesciamo due liste concatenate, le due parti compaiono in ordine inverso. Questo lemma servirà nella dimostrazione della proprietà finale.

```
attribute [local simp] lecture08_concatena_vuota_destra

theorem lecture08_rovescia_concatena
    α{ : Type} (xs ys : Lista α) :
    rovescia (concatena xs ys) =
```

```
      concatena (rovescia ys) (rovescia xs) := by
  induction xs with
  | vuota =>
      simp [concatena, rovescia]
  | cons x xs ih =>
      simp [concatena, rovescia, ih, lecture08_concatena_assoc]
```

Nel caso `vuota`, `simp` usa automaticamente il lemma locale sulla concatenazione con la lista vuota. Nel caso `cons`, l'assunzione induttiva tratta la coda e `lecture08_concatena_assoc` dispone nello stesso modo le concatenazioni sui due lati.

Anche il lemma sul rovesciamento di una concatenazione descrive una forma che vogliamo usare automaticamente nella proprietà finale. Lo aggiungiamo quindi al simp-set locale:

```
attribute [local simp] lecture08_rovescia_concatena
```

1.9 Rovesciare due volte

Possiamo ora dimostrare la proprietà iniziale. L'induzione segue i due costruttori di `Lista α`. Nel caso `cons`, il lemma precedente descrive il rovesciamento della concatenazione prodotta dall'algoritmo e l'assunzione induttiva semplifica il doppio rovesciamento della coda.

```
attribute [local simp] lecture08_concatena_vuota_destra
   lecture08_rovescia_concatena

theorem lecture08_rovescia_rovescia
    α{ : Type} (xs : Lista α) :
    rovescia (rovescia xs) = xs := by
  induction xs with
  | vuota =>
      simp [rovescia]
  | cons x xs ih =>
      simp [rovescia, ih, concatena]
```

La dimostrazione segue la stessa struttura dei programmi: tratta la lista vuota e il caso in cui aggiungiamo una testa a una coda. L'induzione permette di concludere per ogni lista, anche senza conoscerne gli elementi o la lunghezza.

Anche una funzione che restituisse sempre la lista ricevuta soddisferebbe l'equazione del doppio rovesciamento. Questa proprietà, tuttavia, non è una specifica completa di `rovescia`. Per descrivere il comportamento dell'algoritmo servono altre proprietà, come quella che collega `rovescia` e `concatena`, oppure una che stabilisca come cambia la posizione degli elementi.

1.10 Altra automazione: decide e omega

In questi esempi usiamo `Nat` e `List` della libreria standard. `[2, 4, 6]` è la sintassi Lean per una `List Nat`, distinta dalla nostra `Lista`. Vogliamo verificare una concatenazione concreta e una disuguaglianza. `decide` calcola la concatenazione, confronta le liste e verifica che `7 < 10`. Entrambi i controlli danno esito positivo, quindi Lean chiude il goal.

```
example : ([2, 4] ++ [6] : List Nat) = [2, 4, 6] ∧ (7 : Nat) < 10 := by
  decide
```

Eva ha almeno 18 anni e Marta ha tre anni più di Eva. Vogliamo dimostrare che Marta ha almeno 21 anni. Non fissiamo le loro età, ma usiamo le due assunzioni per ricavare il risultato. `omega` tratta l'aritmetica lineare su `Nat` e `Int` (ovvero, uguaglianze e disuguaglianze con somme e prodotti per costanti, senza prodotti fra due variabili).

```
example (etaEva etaMarta : Nat)
    (hDifferenza : etaEva + 3 = etaMarta) (hEta : 18 ≤ etaEva) :
    21 ≤ etaMarta := by
  omega
```

`omega` non è un solver generale per prodotti come `x * y`, con entrambi i fattori variabili.

Con variabili sconosciute, il calcolo usato da `decide` può bloccarsi. Per esempio, in `n < n + 1` non abbiamo due numeri concreti da confrontare. `omega` può invece ragionare su questi vincoli e dimostrare la disuguaglianza per un naturale arbitrario.

1.11 Un LLM come assistente per Lean

Un LLM, dall'inglese *large language model*, è un modello linguistico al quale possiamo chiedere spiegazioni o codice in linguaggio naturale. Qui lo usiamo per proporre una dimostrazione a partire dalle definizioni e dai lemmi del nostro file. La proposta può contenere errori, perciò la leggiamo e la facciamo controllare a Lean.

1.11.1 Installare Codex in VS Code

Usiamo l'estensione ufficiale Codex su Windows, macOS o Linux. Apri il progetto del corso in VS Code e controlla che Lean e l'Infoview funzionino. Serve un account ChatGPT con accesso a Codex. Disponibilità e limiti dipendono dall'account.

1. Apri *Extensions / Estensioni*: `Ctrl+Shift+X` su Windows e Linux, `Cmd+Shift+X` su macOS.

2. Cerca `@id:OpenAI.chatgpt`. Verifica il nome *Codex* e il publisher *OpenAI*. Puoi anche aprire la pagina ufficiale dell'estensione.

3. Premi *Install / Installa*. Se richiesto, ricarica la finestra.

4. Premi l'icona Codex nella barra laterale. Se non compare, apri la *Command Palette* con `Ctrl+Shift+P` su Windows e Linux o `Cmd+Shift+P` su macOS. Cerca ed esegui *Codex: Open Codex Sidebar*.

Per questa attività basta l'estensione, senza installare anche l'app desktop o la CLI di Codex. Anche su Windows possiamo usarla direttamente in VS Code. Manteniamo il progetto nella finestra in cui Lean funziona già.

1.11.2 Accedere con ChatGPT

1. Nel pannello Codex scegli *Sign in with ChatGPT*.

2. Nel browser completa l'accesso con il tuo account ChatGPT.

3. Torna a VS Code. Se richiesto, consenti al browser di riaprire l'editor.

4. Verifica che sia disponibile il campo per iniziare una conversazione.

Le istruzioni aggiornate sono nella documentazione ufficiale di OpenAI sull'estensione per IDE, sull'accesso e sull'ambiente Windows.

1.11.3 Iniziare dal file Lean

Salva il file e includi nella conversazione l'enunciato su cui vuoi lavorare, controllando il contesto mostrato da Codex. Una prima richiesta può essere:

«Spiega questo enunciato e il ruolo dei suoi parametri. Usa soltanto i concetti già presenti nel file. Per ora non modificare file e non eseguire comandi.»

Leggi la risposta confrontandola con il codice. Quando chiedi modifiche, esamina le differenze e controlla il risultato con Lean. Codex propone spiegazioni e codice, mentre Lean controlla le dimostrazioni.

1.11.4 Chiedere una dimostrazione

Vogliamo dimostrare che, se due liste rovesciate sono uguali, anche le liste originali sono uguali. L'assunzione è `h : rovescia xs = rovescia ys`, e il goal è `xs = ys`.

Una richiesta all'assistente può specificare sia il risultato desiderato sia i vincoli sulla dimostrazione:

«Spiega prima una strategia per questo enunciato. Mantieni definizioni, assunzioni e conclusione. Usa i lemmi già presenti. Non aggiungere assiomi, `sorry` o `admit`. Poi proponi una dimostrazione per passi e controllala con Lean.»

Controlliamo la proposta in Lean. Se c'è un errore, forniamo all'assistente il messaggio e il goal rimasto per chiedere una correzione. Possiamo anche chiedergli di ripetere il controllo e le correzioni finché Lean non accetta la dimostrazione, modificando soltanto la dimostrazione e lasciando invariato l'enunciato del teorema. Restano i vincoli precedenti: non aggiungere assiomi, `sorry` o `admit`.

1.11.5 Dimostrazione

Riscriviamo `xs` come `rovescia (rovescia xs)`, usando al contrario il teorema sul doppio rovesciamento. Poi usiamo `h` per sostituire `rovescia xs` con `rovescia ys`. Il goal diventa `rovescia (rovescia ys) = ys`, che segue dallo stesso teorema.

```
theorem lecture08_rovescia_iniettiva α{ : Type} (xs ys : Lista α)
    (h : rovescia xs = rovescia ys) : xs = ys := by
  -- Riscriviamo xs come il suo doppio rovesciamento.
  rw ←[ lecture08_rovescia_rovescia xs]
  -- Usiamo h per sostituire rovescia xs con rovescia ys.
  rw [h]
  -- Il doppio rovesciamento di ys restituisce ys.
  rw [lecture08_rovescia_rovescia ys]
```

Concludiamo l'attività confrontando enunciato e definizioni con quelli iniziali, leggendo la dimostrazione e controllando il file con Lean.

`#print axioms lecture08_rovescia_iniettiva` mostra le dipendenze assiomatiche. Per questa dimostrazione, nell'ambiente del corso compare `propext`, l'assioma standard che identifica proposizioni equivalenti. Non compare `sorryAx`, che segnalerebbe una dimostrazione incompleta.

La dipendenza da `propext` proviene dai lemmi precedenti dimostrati con `simp`. Per esempio, il semplificatore usa `eq_self` per sostituire la proposizione `t = t` con `True`. Considerare uguali queste due proposizioni equivalenti richiede `propext`. Il teorema finale eredita la dipendenza tramite `lecture08_rovescia_rovescia`, anche se la sua dimostrazione contiene soltanto riscritture. L'elenco riguarda la dimostrazione scritta: un'altra dimostrazione dello stesso enunciato potrebbe usare principi diversi.

Il kernel è la parte di Lean che controlla la dimostrazione prodotta, anche quando l'abbiamo ottenuta tramite tattiche o un assistente. Verifica che dimostri l'enunciato scritto. Spetta a noi controllare che quell'enunciato esprima la proprietà desiderata.

1.12 Fonti

- Avigad, de Moura, Kong e Ullrich, Theorem Proving in Lean 4, Axioms and Computation: https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/.
- Lean project, *The Lean Language Reference*, «Tactic Reference»: https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/.
- Lean project, *The Lean Language Reference*, «The grind tactic»: https://lean-lang.org/doc/reference/latest/The--grind--tactic/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong e Sebastian Ullrich, *Theorem Proving in Lean 4*, «Inductive Types»: https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong e Sebastian Ullrich, *Theorem Proving in Lean 4*, «Induction and Recursion»: https://lean-lang.org/theorem_proving_in_lean4/Induction-and-Recursion/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong e Sebastian Ullrich, *Theorem Proving in Lean 4*, «Tactics»: https://lean-lang.org/theorem_proving_in_lean4/Tactics/.
- David Thrane Christiansen, *Functional Programming in Lean*, «Datatypes and Patterns»: https://lean-lang.org/functional_programming_in_lean/Getting-to-Know-Lean/Datatypes-and-Patterns/.
- Lean project, *The Lean Language Reference*, «Recursive Definitions»: https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.

- Kevin Buzzard e Jon Eugster, *Natural Number Game*: https://github.com/leanprover-community/NNG4.
- Vrije Universiteit Amsterdam, *Logical Verification*: https://research.vu.nl/en/courses/logical-verification-5/.
- Marieke Huisman e Anton J. Wijs, *Concise Guide to Software Verification: From Model Checking to Annotation Checking*: https://research.utwente.nl/en/publications/concise-guide-to-software-verification-from-model-checking-to-ann/.
- Hana Chockler, Joseph Y. Halpern e Orna Kupferman, *What Causes a System to Satisfy a Specification?*: https://arxiv.org/abs/cs/0312036.
- Lean project, The Lean Language Reference, The Simplifier: https://lean-lang.org/doc/reference/latest/The-Simplifier/.
- Lean project, Lean 4.29.1, Init/Tactics.lean: https://github.com/leanprover/lean4/blob/v4.29.1/src/Init/Tactics.lean.